ADVANCE PROGRAMMING (OOPS) LAB

PCCCS495

PROJECT REPORT

NAME:- Parijat Bhattacharya

STREAM:- CSE(IoT) **SEMESTER:-** 4th

ENROLLMENT NO:- 12024052019011

CLASS ROLL NO:- 16

PROJECT TITLE:- Dynamic Rental Management System

INSTRUCTOR NAME:- Dr. Soumadip Biswas and Dr. Swagatam Basu

Spring 2026

PROBLEM STATEMENT :-

Rental businesses manage different types of assets such as vehicles, rooms, or equipment. Each asset category follows different pricing rules, late fee policies, and availability constraints. Managing these variations manually or with a rigid system leads to certain errors, inefficiency, and difficulty in scaling. This system will dynamically calculate rental costs, manage availability, enforce return policies, and handle exceptional scenarios such as late returns or invalid bookings. By applying core Object Oriented principles, this system ensures scalability, and maintainability while maintaining clean separation between business logic and data management.

Major Problem - Managing rental systems manually can lead to errors in tracking items, customers, and transactions.

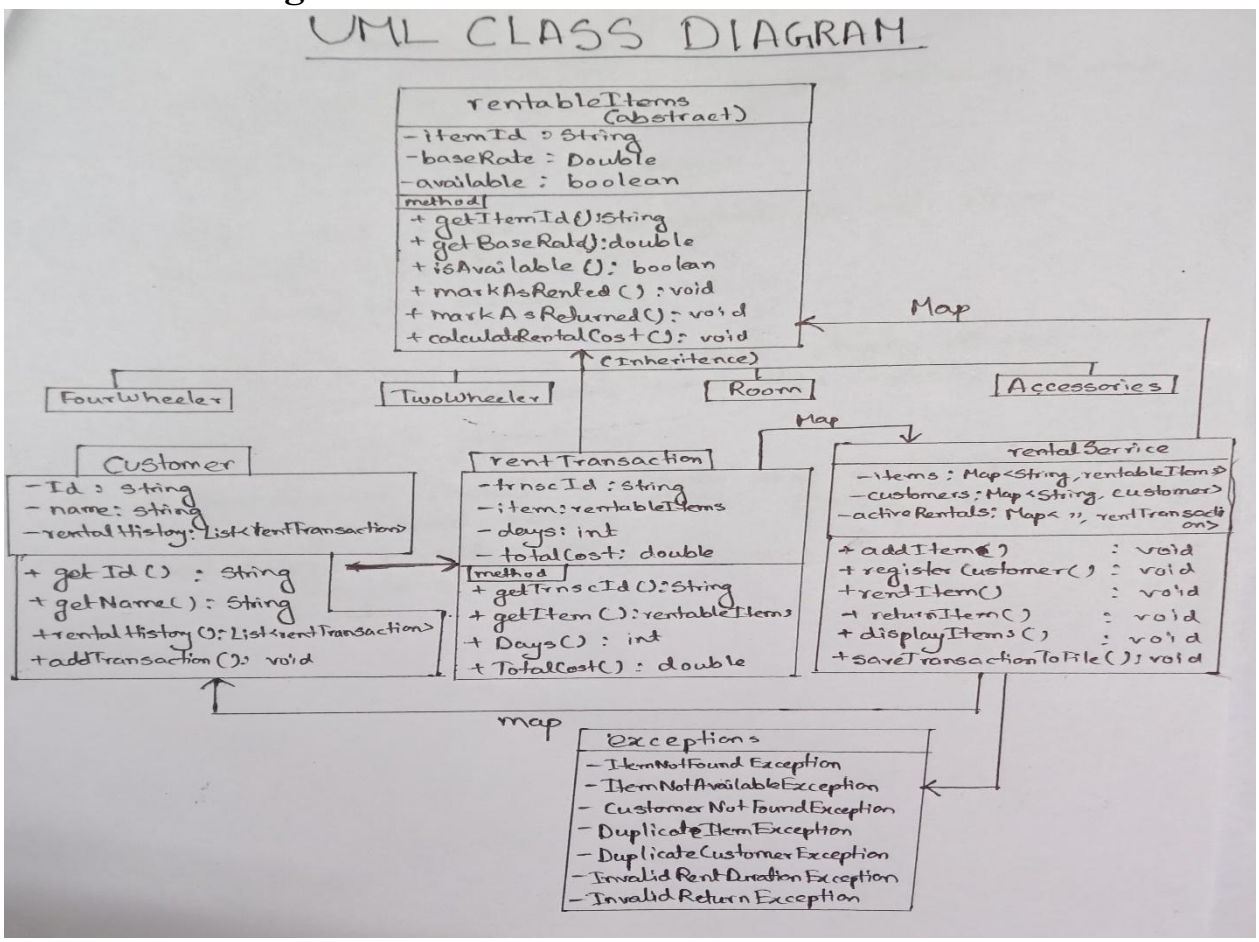
Solution - This project implements a dynamic rental management system using Java and OOP principles.

Features –

- Item management
- Customer handling
- Rental transactions
- File-based persistence

SYSTEM DESIGN :-

UML Class Diagram :-



Description of major classes :-

- **rentableItems**
  - Abstract base class representing all rentable entities.
- **FourWheeler, TwoWheeler, Room, Accessories**
  - Subclasses implementing specific pricing logic.
- **customer**
  - Stores customer details and transaction history.
- **rentTransaction**
  - Represents a rental with cost calculation.
- **rentalService**
  - Core class handling all operations and file persistence.

Package structure:-

-com.parijat.rental

- ↳ model
- ↳ service
- ↳ exception
- ↳ main

Interaction flow :-

User → main → rentalService → Model Classes → CSV file

OOP CONCEPTS DEMONSTRATED:-

Abstraction:-

- Used: rentableItems
- Used because rentableItems defines common attributes and the abstract method calculateRentalCost() i.e. to provide a common blueprint for all rentable items without fixing the implementation details.
- Snippet-

```
public abstract class rentableItems{
private String itemId;
private double baseRate;
…….
public abstract double calculateRentalCost(int days); }
```

- Explanation- Abstraction allows defining what an object should do without specifying how it does it. Here, all items must calculate rental cost, but each type implements it differently.

Encapsulation:-

- Used: used for private variables
- Used for the purpose to protect internal data and prevent direct modification from outside the class.
- Snippet-

```
private List<rentTransaction> rentalHistory;
…….
public List<rentTransaction> rentalHistory() {
```

```
        return new ArrayList<>(rentalHistory);
    }
```

- Explanation- This code demonstrates encapsulation by keeping the rentalHistory list private and restricting direct access. Instead of returning the original list, a copy is returned to prevent external modification. The addTransaction() method provides controlled access to update the data, ensuring data safety.

Inheritance:-

- Used: Classes like FourWheeler, TwoWheeler, Room, and Accessories extend RentableItem.
- Used for the purpose to reuse common properties and methods defined in the base class.
- Snippet-

```java
public class FourWheeler extends rentableItems {
    public FourWheeler(String itemId, double baseRate) {
        super(itemId, baseRate);
    }
    @Override
    public double calculateRentalCost(int days) {
        return getBaseRate()*days+150; //150 fixed maintanance cost for four wheeler
    }
}
```

- Explanation- this snippet explains reduction of code duplication and creates a clear hierarchy. All item(like fourwheeler here) types share basic properties but can have different behaviors.

Polymorphism:-

- Used: used in the calculateRentalCost(int days) method, which is defined in the abstract class RentableItem and overridden in its subclasses like FourWheeler, TwoWheeler, Room, and Accessories. It is invoked in the rentItem() method of the RentalService class.
- Used for the purpose to allow a single method call to behave differently depending on the type of item being rented.
- Snippet-

```
public abstract double calculateRentalCost(int days);    ← In rentableItems

this.totalCost = item.calculateRentalCost(days);
                                                         ← In rentTransaction
```

- Explanation- The variable item is of type RentableItem, but it can refer to different subclass objects at runtime. When calculateRentalCost() is called, the appropriate implementation is executed based on the actual object type (e.g., FourWheeler or Room). This is an example of runtime polymorphism.

Exception Handling:-

- Used: Custom exceptions are thrown in the rentalService class.
- Used to handle errors in a structured and meaningful way
- Snippet-

```
if (days<=0){ throw new
InvalidRentDurationException("Days must be greater than
zero");
 }
```

- Explanation- this explains that if days are less than 0, it will throw custom exception which is predefined.

Collection:-

- Used: HashMap and ArrayList are used to store items, customers, and transactions.
- Used to efficiently store and retrieve data using keys.
- Snippet-

```
private Map<String, RentableItem> items = new
HashMap<>();
```

- Explanation- This line declares a HashMap that stores all rentable items using a unique String ID as the key and the corresponding RentableItem object as the value. It allows efficient storage and quick retrieval of items based on their IDs, which is useful for operations like searching, renting, and updating items.

IMPLEMENTATION HIGHLIGHTS:-

1. Rental Processing Logic:-

```
rentTransaction transaction =new rentTransaction(transactionId, item, days);
System.out.println("Total Rental Cost: " + transaction.TotalCost());
item.markAsRented();
activeRentals.put(transactionId, transaction);
customer customer =customers.get(customerId);
customer.addTransaction(transaction);
```

This snippet represents the core workflow of the system. It retrieves the item,

creates a transaction, updates availability, stores the transaction, and records it for the customer. It ensures that all necessary steps of the rental process are handled in a structured manner.

2. HashMap Usage:-

private Map<String, RentableItem> items = new HashMap<>();

HashMap is used to store items with their unique IDs as keys, allowing fast retrieval and efficient management. This avoids the need for searching through lists and improves the overall performance of the system.

3. File Handling for Persistence:-

```
try {
    FileWriter fw = new FileWriter("transactions.csv", true);
    fw.write(transaction.getTrnscId() + "," +
        transaction.getItem().getItemId() + "," +
        transaction.Days() + "," +
        transaction.TotalCost() + "\n");

    fw.close();
}catch (IOException e){
    System.out.println("File error: " + e.getMessage());
}
```

This snippet implements file-based persistence by storing transaction details in a CSV file. The use of append mode ensures that previous data is not overwritten, allowing continuous tracking of rental records.

4. Exception Throwing for Validation:-

```
public void rentItem(String transactionId, String customerId, String itemId, int days)
        throws CustomerNotFoundException, ItemNotFoundException,
                ItemNotAvailableException, InvalidRentDurationException{
    if (days<=0) {
        throw new InvalidRentDurationException(message: "Days must be greater than zero");
    }
    if (!customers.containsKey(customerId)) {
        throw new CustomerNotFoundException(message: "Customer not found");
```

This ensures that invalid operations are handled properly. Throwing custom exceptions helps in identifying specific errors and prevents the system from proceeding with incorrect data.

TESTING AND ERROR HANDLING:-

Test Cases Considered:-

1. Add item
2. Register customer
3. Rent item

4. Return item

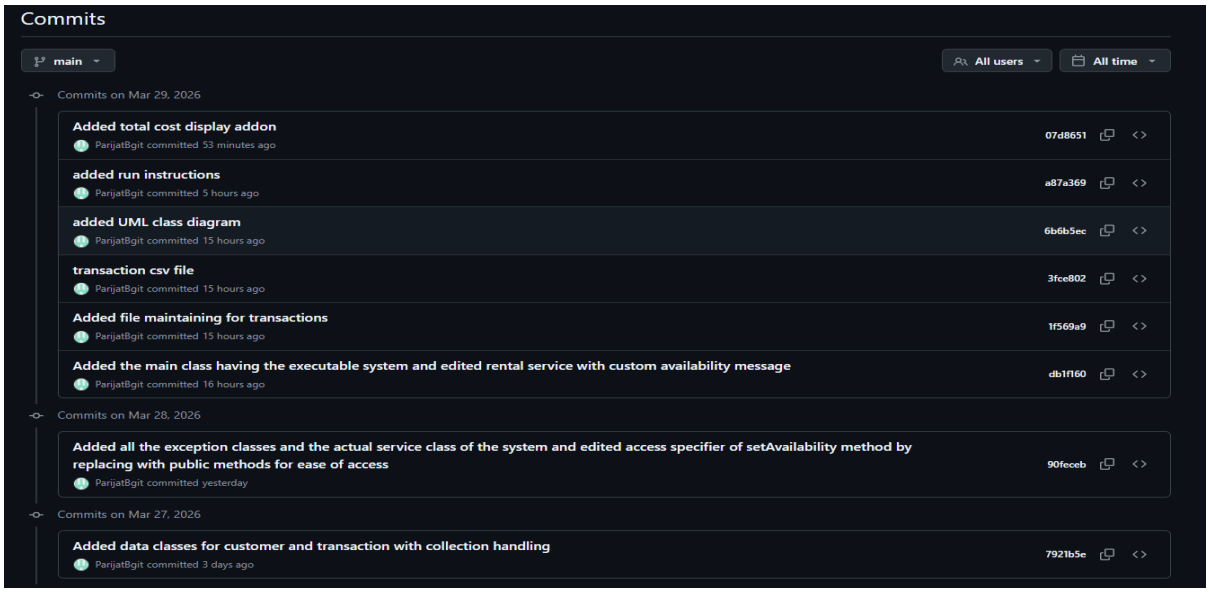
Edge Cases Handled:-

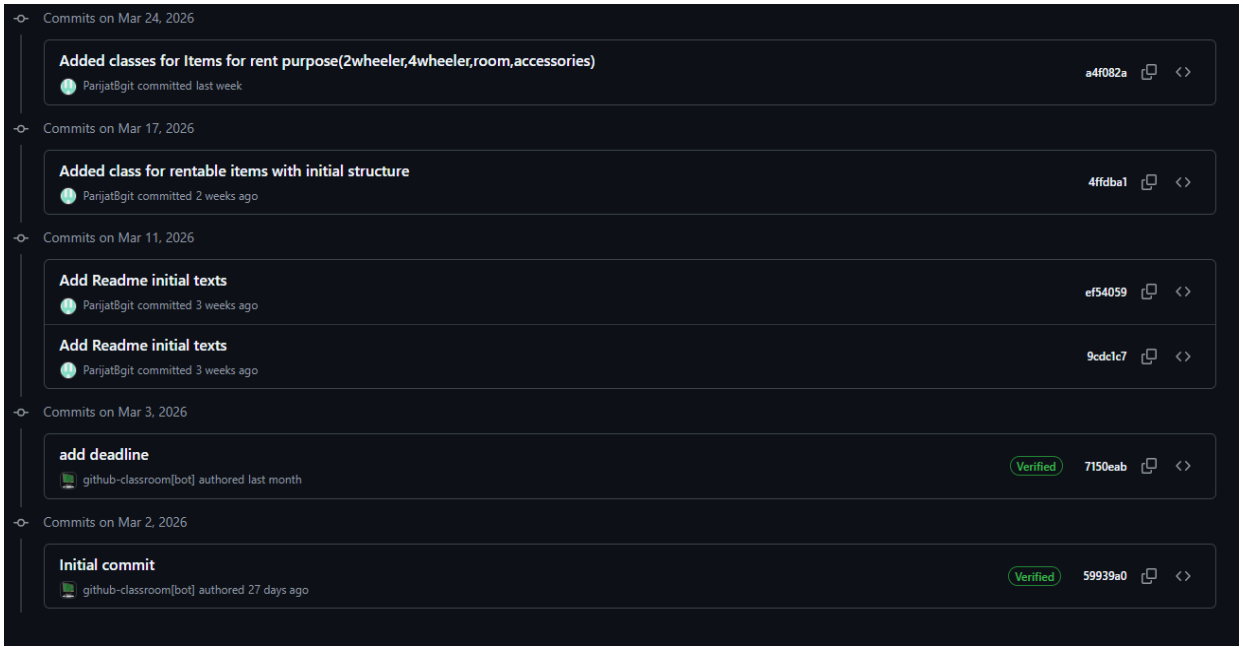
1. Renting unavailable item
2. Invalid days
3. Duplicate entries

Failure Scenarios:-

1. Invalid item ID
2. Invalid transaction ID

GIT WORKFLOW:-





The project was developed incrementally with commits for each major feature such as model creation, service logic, exception handling, and file persistence.

CONCLUSION and FUTURE SCOPE:-

The Dynamic Rental Management System successfully demonstrates the application of core object-oriented programming concepts such as abstraction, encapsulation, inheritance, and polymorphism. The system provides a structured approach to managing rental items, customers, and transactions. Efficient data handling is achieved using collections like HashMap, while custom exception handling ensures robustness against invalid operations. Additionally, file-based persistence using a CSV file allows basic data storage beyond program execution. Overall, the project achieves its objective of building a functional and well-organized rental management system.

The system can be further enhanced by adding several features to improve functionality and usability. A graphical user interface (GUI) can be developed to make the system more user-friendly. Database integration (such as MySQL) can replace file-based storage for better scalability and data management. Features like late fee calculation, user authentication, and admin controls can be introduced to make the system more realistic. Additionally, reporting and analytics features can be added to track rental trends and performance.

APPENDIX:

Initial Project Proposal: **C:\Users\parijat\OneDrive\Desktop\PROJECT PROPOSAL FOR OOPS.pdf**

Copy of Review email: C:\Users\parijat\OneDrive\Desktop\PCCCS495 Proposal Review – APPROVED_MINOR - PARITT - Outlook.pdf